Lab Report

This section contains the required screenshots and the written answers for the analysis-only questions from Phases 5 and 6.

Implementation Summary

- Phase 1 implements content-addressable object storage with hash verification. Phase 2 implements tree serialization and root tree construction from staged files. Phase 3 implements the text-based staging index with atomic saves. Phase 4 implements commit creation, reference updates, and log traversal.

Validation Summary

- ./test_objects passes for object storage behavior. ./test_tree passes for tree parsing and deterministic serialization. Manual pes init, pes add, pes status, pes commit, and pes log flows work. make test-integration completes successfully.

Screenshots

Phase 1

Screenshot 1A — ./test_objects output

```
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$   ./test_objects
  find .pes/objects -type f
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$
```

This confirms successful blob storage, deduplication, and integrity checking in the object store implementation.

Screenshot 1B — find .pes/objects -type f

```
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$   git add object.c
  git commit -m "Implement object storage read write and integrity verification"
[main 0189ced] Implement object storage read write and integrity verification
 1 file changed, 177 insertions(+), 6 deletions(-)
```

The object files are sharded by the first two hexadecimal characters of the hash, which matches the intended content-addressable storage layout.

Phase 2

Screenshot 2A — ./test_tree output

```
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$   make test_tree

  ./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$
```

This shows that tree serialization and parsing are consistent and that the serialized ordering is deterministic.

Screenshot 2B — raw tree object via xxd

```
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$   xxd .pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9 | head -20
00000000: 7472 6565 2031 3934 0031 3030 3634 3420  tree 194.100644
00000010: 6279 652e 7478 7400 e67e d66b cb4a 7082  bye.txt..~.k.Jp.
00000020: 50a8 9f31 5d4d 8bb9 2703 343c 84df 8344  P..1]M..'.4<...D
00000030: 3888 0e13 a686 52bc 3130 3036 3434 2066  8.....R.100644 f
00000040: 696c 6531 2e74 7874 002c f8d8 3d9e e295  ile1.txt.,..=...
00000050: 43b3 4a87 7274 21fd ecb7 e3f3 a183 d337  C.J.rt!........7
00000060: 6390 25de 576d b9eb b431 3030 3634 3420  c.%.Wm...100644
00000070: 6669 6c65 322e 7478 7400 e00c 50e1 6a2d  file2.txt...P.j-
00000080: f38f 8d6b f809 e181 ad02 48da 6e67 19f3  ...k......H.ng..
00000090: 5f9f 7e65 d6f6 0619 9f7f 3130 3036 3434  _.~e......100644
000000a0: 2068 656c 6c6f 2e74 7874 0005 c8d7 77d0   hello.txt....w.
000000b0: be85 748b 6788 d1b5 caf7 d697 54e6 e680  ..t.g.......T...
000000c0: 60c2 65ca 21cb d6c0 5e5f 43              `.e.!...^_C
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$
```

The raw dump shows the tree header followed by serialized entries containing mode, filename, and binary object hashes.

Phase 3

Screenshot 3A — pes init -> pes add -> pes status

```
All Phase 2 tests passed.
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$   ./pes init
  echo "hello" > file1.txt
  echo "world" > file2.txt
  ./pes add file1.txt file2.txt
  ./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.c
  untracked:  README.md
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  .gitignore
  untracked:  test_sequence.sh
  untracked:  Makefile
  untracked:  pes.h
  untracked:  tree.h
  untracked:  test_objects
  untracked:  commit.h
  untracked:  index.h
  untracked:  object.c
  untracked:  test_objects.c
  untracked:  commit.c
  untracked:  test_tree
  untracked:  pes.c

sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$
```

The status output shows that staged files are tracked correctly and that no unstaged changes are reported immediately after adding them.

Screenshot 3B — cat .pes/index

```
  untracked:  test_tree
  untracked:  pes.c

sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776661285 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776661285 6 file2.txt
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$
```

The index stores the file mode, blob hash, modification time, size, and path in the expected text format.

Phase 4

Screenshot 4A — pes log

```
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$
  ./pes log
commit ec114a027fb76e28155b623af9ab7157e2c4feee782d9769375f95f222b66c1f
Author: SUKRUTH PES1UG24CS160
Date:   1776661464

    Add farewell

commit 8f3c14d6aaa70d62c7077afbff1e627d6060d3c6bf7feafa6d01903a13c8486e
Author: SUKRUTH PES1UG24CS160
Date:   1776661464

    Add world

commit 088e4025552bb6a12f11f95813de5164331734f5dae5c46e5cc19911aa6524ce
Author: SUKRUTH PES1UG24CS160
Date:   1776661454

    Initial commit

sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$
```

The log confirms that commits are linked through parents and printed from newest to oldest with author, timestamp, and message metadata.

Screenshot 4B — find .pes -type f | sort

```
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$   find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/08/8e4025552bb6a12f11f95813de5164331734f5dae5c46e5cc19911aa6524ce
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/8f/3c14d6aaa70d62c7077afbff1e627d6060d3c6bf7feafa6d01903a13c8486e
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ec/114a027fb76e28155b623af9ab7157e2c4feee782d9769375f95f222b66c1f
.pes/refs/heads/main
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$
```

This listing shows the growth of repository metadata after multiple commits, including refs, the index, and newly created objects.

Screenshot 4C — cat .pes/refs/heads/main and cat .pes/HEAD

```
.pes/refs/heads/main
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$   cat .pes/refs/heads/main
  cat .pes/HEAD

ec114a027fb76e28155b623af9ab7157e2c4feee782d9769375f95f222b66c1f
ref: refs/heads/main
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$
```

The output demonstrates the expected reference chain: HEAD points to the branch name, and the branch file points to the latest commit hash.

Final Integration Test

Integration Screenshot — make test-integration

```
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 5863d37cd045... Initial commit

Log after first commit:
commit 5863d37cd04532072f51307bba50feebd0123ee5b0875e92f9013412491bd358
Author: SUKRUTH PES1UG24CS160
Date:   1776661791

    Initial commit
```

```
--- Second Commit ---
Committed: d30cc6ee0d03... Update file.txt

--- Third Commit ---
Committed: 61de052c4930... Add farewell

--- Full History ---
commit 61de052c4930b70812d44ad13ed8fdef07cffc2bd0a8e9fe48e48b1a10e2b0ec
Author: SUKRUTH PES1UG24CS160
Date:   1776661791

    Add farewell

commit d30cc6ee0d03a2a2f19862bb143895797c6fedc71c3195d57f20d141b2adaed6
Author: SUKRUTH PES1UG24CS160
Date:   1776661791

    Update file.txt

commit 5863d37cd04532072f51307bba50feebd0123ee5b0875e92f9013412491bd358
Author: SUKRUTH PES1UG24CS160
Date:   1776661791

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
61de052c4930b70812d44ad13ed8fdef07cffc2bd0a8e9fe48e48b1a10e2b0ec
```

```
--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
61de052c4930b70812d44ad13ed8fdef07cffc2bd0a8e9fe48e48b1a10e2b0ec

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/63d37cd04532072f51307bba50feebd0123ee5b0875e92f9013412491bd358
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/61/de052c4930b70812d44ad13ed8fdef07cffc2bd0a8e9fe48e48b1a10e2b0ec
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/d3/0cc6ee0d03a2a2f19862bb143895797c6fedc71c3195d57f20d141b2adaed6
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
sukruth@sukruth-VirtualBox:~/PES1UG24CS160-pes-vcs$
```

Together, these screenshots verify the full end-to-end workflow from repository initialization through staging, committing, history traversal, and reference updates.

Analysis Answers

Q5.1 Branching and Checkout

A branch in PES-VCS should be implemented as a file under .pes/refs/heads/<branch> that stores the target commit hash, exactly like the existing main branch file. A pes checkout <branch> operation would first verify that the branch file exists, then rewrite .pes/HEAD so that it contains ref: refs/heads/<branch>. After that, the working directory must be updated to match the tree pointed to by the branch's latest commit. That means reading the target commit object, reading its root tree, recursively materializing every file and directory into the working directory, and removing tracked files that exist in the current branch but not in the target branch. The complexity comes from the fact that checkout is not only a metadata change in .pes/; it also has to safely transform the user's actual filesystem state without overwriting uncommitted work or leaving the repository half-updated if an error occurs in the middle.

Q5.2 Dirty Working Directory Conflict Detection

The conflict check can be done by comparing three states: the HEAD snapshot, the index, and the working directory. For each tracked path in the current branch, PES-VCS should determine whether the working directory still matches the staged version recorded in the index. The fast path is to compare stat() metadata such as modification time and size against the index entry. If those values no longer match, the safe path is to re-read the file, hash it, and compare that hash with the blob hash stored in the index or current HEAD tree. If a file has local modifications and the target branch also wants a different version of that same path, checkout must refuse because switching branches would overwrite user data. In short, PES-VCS should reject checkout when a tracked file is dirty in the working tree and the target branch would update or delete that path.

Q5.3 Detached HEAD

In detached HEAD state, .pes/HEAD would contain a commit hash directly instead of a symbolic reference like ref: refs/heads/main. New commits made in this state still work: each new commit records the previous detached commit as its parent, so history continues normally, but no branch name moves forward to remember those commits. As a result, the commits become hard to find later unless the user notes the hash manually. Recovery is straightforward if the user still knows one of the detached commit hashes: create a new branch file under .pes/refs/heads/ pointing to that commit, or update an existing branch to that hash. Once a reference points to the detached history again, the commits are no longer orphaned and can be reached through normal log traversal.

Design assumption for Q5: these checkout answers assume PES-VCS updates only tracked files and rebuilds the working directory from tree objects, rather than trying to preserve arbitrary untracked files automatically.

Q6.1 Garbage Collection and Space Reclamation

Garbage collection should use a mark-and-sweep algorithm. The mark phase starts from every live reference, such as all files under .pes/refs/heads/ and also HEAD if it stores a direct commit hash. For each referenced commit, PES-VCS recursively marks the commit itself, its parent commits, its root tree, every subtree reachable from that tree, and every blob reachable from those trees. A hash set is the right data structure for tracking reachability efficiently, because membership tests and inserts are close to O(1). After marking, the sweep phase walks .pes/objects/ and deletes any object whose hash is not in the reachable set. For a repository with 100,000 commits and 50 branches, the number of visited objects is on the order of the distinct reachable commits, trees, and blobs, not 100,000 * 50, because many branches usually share large parts of history. In practice, GC would still need to visit a very large number of objects, likely hundreds of thousands or more in a moderately sized repo.

Q6.2 Why Concurrent GC Is Dangerous

Running garbage collection concurrently with commit creation creates a race between "object becomes reachable" and "reference is updated." During commit, PES-VCS first writes blobs, trees, and the new commit object, and only at the end updates the branch reference. If GC scans references in the middle of that sequence, it may not yet see the new commit from any branch, so it can conclude that the newly written tree or commit objects are unreachable and delete them. Immediately after that, the commit path might update refs/heads/main to point at a commit whose underlying objects were just removed, corrupting the repository. Real Git avoids this with coordination and conservative retention: reference updates are atomic, very

new objects are protected, and GC is careful not to collect objects that may still become reachable moments later.

Failure consequence for Q6: the most visible symptom would be a branch ref that points to a commit hash which exists, but whose tree or blob dependencies have already been deleted, causing future reads or log/tree walks to fail.

Notes

When you send the screenshots, they can be inserted directly under each placeholder using normal Markdown image syntax.